

Lab0: GitLab

熊思宇 24307130130

一、任务一

1. 多人协同开发经历

我有过一些多人协同开发的经历，主要是在课程的小组作业中。当时通常先由小组讨论确定任务分工，再由每个人独立完成自己负责的部分，最后汇总成果进行对比和整合。我在这一过程中没有使用 Git 等版本控制工具，代码合并与版本统一主要依赖人工沟通和手动整理。这种方式在任务较少时能够完成协作，但修改次数增多后，效率不高，也容易发生版本混乱和修改相互覆盖。通过学习 Git，我认识到版本控制能够记录每次修改，并通过分支与合并机制支持多人协作，减少仅靠人工管理文件带来的问题。

2. 为什么要设计暂存与提交两个步骤

Git 将暂存（stage）与提交（commit）分开，可以让开发者精确控制每次提交的内容。工作区保存正在编辑的文件，暂存区保存准备纳入下一次提交的内容，而提交将暂存区中的内容记录为一个新的版本，形成一个完整全面的工作流。

+暂存区支持选择性提交。如果我同时修改了多个文件，而这些修改属于不同任务，就可以用 `git add` 只暂存与本次任务相关的内容，让其他修改继续保留在工作区中。同时，暂存区的存在也提供了提交前的检查机会，可以降低出现错误的可能性。

3. `git branch` 与 `git branch -a` 的区别

`git branch` 只列出本地分支，通常用星号标记当前所在分支。`git branch -a` 表示 `git branch --all`，除了本地分支，还会显示远程跟踪分支，例如 `remotes/origin/main`。远程跟踪分支是本地保存的远程分支状态记录，并不一定与远程仓库的实时状态一致。执行 `git fetch` 可以更新这些记录，再使用 `git branch -a` 查看。

二、任务三

1. 《Commit message 和 Change log 编写指南》

规范的 Commit Message 能清楚记录代码修改的目的，方便浏览项目历史、查找特定类型的提交，文章介绍了一种统一的提交说明格式：

`<type>(<scope>): <subject>`

其中，type 表示提交类型，如 `feat` 表示新增功能、`fix` 表示修复问题、`docs` 表示文档修改、`style` 表示格式调整、`refactor` 表示重构、`test` 表示测试改动、`chore` 表示构建或辅助工具改动。scope 表示修改影响的范围，subject 简要说明提交目的。需要进一步解释时，可以在 Body 中说明修改原因和修改前后的差异，在 Footer 中记录不兼容变更或关联的 Issue。

统一格式可以让提交历史更容易阅读，同时利用 Change log 能够按类型整理新增

功能、问题修复和不兼容变更，减少发布版本时手动整理更新日志的工作。

2. 《Gitflow 使用规范》

文章介绍了 Gitflow 的分支版本管理系统，通过不同分支组织开发、发布和维护的工作流程，保证不同分支各司其职，避免多人协作过程中代码冲突、代码版本出现问题。

在具体的工作流程中，main（或 master）保存正式发布的稳定代码，develop 汇总下一版本的开发成果，feature 用于开发新功能，release 用于发布前的测试与修复，hotfix 用于紧急修复正式版本的问题。

开发新功能时，从 develop 创建 feature 分支，完成后合并回 develop。准备发布时，从 develop 创建 release 分支，完成测试修复后合并到正式分支和 develop。线上出现紧急问题时，从正式分支创建 hotfix 分支，修复后将修改同步到正式分支和开发分支；如果存在正在准备的发布分支，也需要按流程同步修复。这种流程通过明确各类分支的职责，让功能开发、版本准备和线上修复可以相对独立地进行。

2.对为什么要学习 Git 的理解

对于自己的个人项目：Git 可以帮助我更好地管理项目中的文件变化，平时写代码时，经常会遇到改错内容、误删文件，或者修改后想恢复旧版本的情况，如果不断复制文件夹，很容易造成混乱，也很难判断每个版本具体改了什么，Git 可以记录每次修改的内容、时间和提交者，需要时还能查看历史记录或恢复到之前的状态，使项目管理更加清晰。

对于团队协作办公：多人共同完成一个项目时，如果大家直接修改同一份文件，很容易出现内容被覆盖或者版本不一致的问题。Git 允许每个人在自己的分支上完成任务，之后再把修改合并到主分支。这样既不会轻易影响其他人的工作，也方便检查不同成员分别做了哪些改动。Git 的分支创建和切换都比较方便，因此适合同时开发新功能、修复问题和维护稳定版本。

还有就是，GitHub 这种实用网站平台以 Git 为基础，代码提交、项目协作和版本发布往往离不开它。

三、实验步骤和过程

1. 修改 main.c 并提交（任务 2）

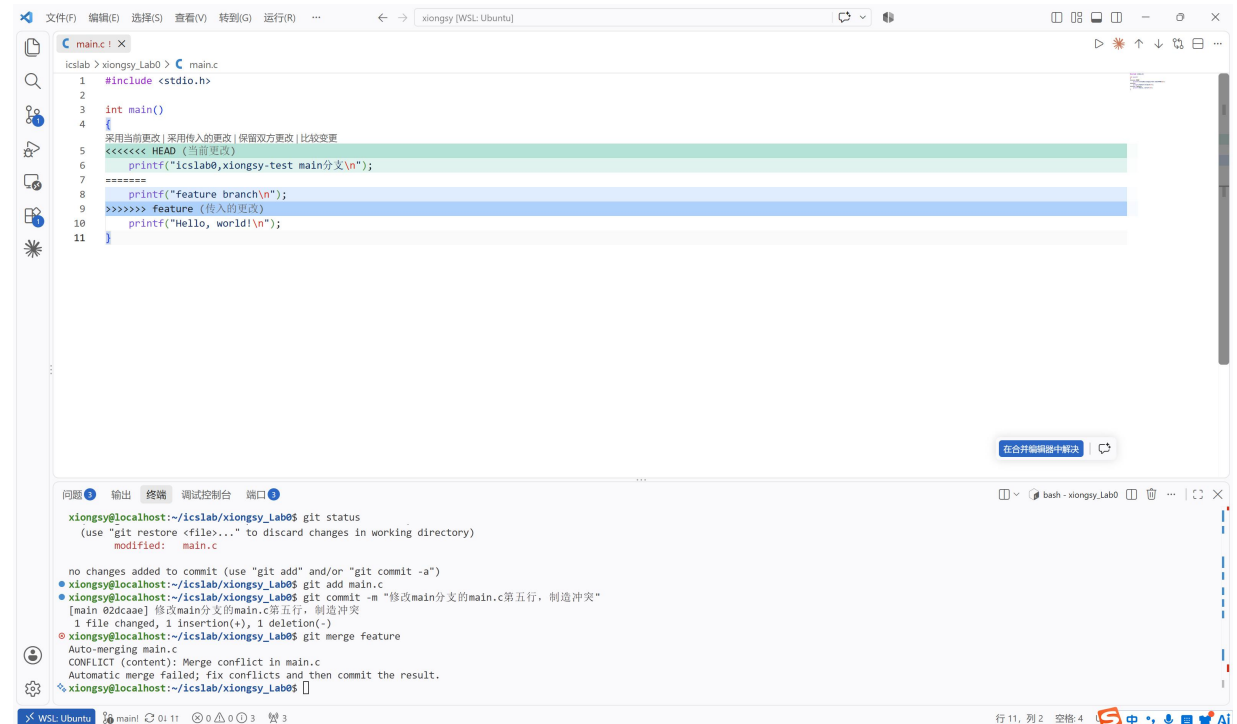
```
icslab > xiongsy_Lab0 > C main.c
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("icslab0,xiongsy-test main分支\n");
6      printf("Hello, world!\n");
7  }
```

完成并提交了 main.c 中 TODO 部分的字符串修改

2. 分支修改与合并冲突（任务 4）

我在 feature 和 main 分支上分别修改了 main.c 第 5 行的 printf 语句，并提交了不同的内容。在 main 分支执行 git merge feature 时，Git 报告 main.c 存在内容冲突，需要手动处理。



```
icslab > xiongsy_Lab0 > C main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6     <<<<<< HEAD (当前更改)
7     printf("icslab0,xiongsy-test main分支\n");
8     =====
9     printf("feature branch\n");
10    >>>>>> feature (传入的更改)
11    printf("Hello, world!\n");
12 }
```

```
xiongsy@localhost:~/icslab/xiongsy_Lab0$ git status
(use "git restore <file>..." to discard changes in working directory)
modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
xiongsy@localhost:~/icslab/xiongsy_Lab0$ git add main.c
xiongsy@localhost:~/icslab/xiongsy_Lab0$ git commit -m "修改main分支的main.c第五行，制造冲突"
[main 62dcaae] 修改main分支的main.c第五行，制造冲突
1 file changed, 1 insertion(+), 1 deletion(-)
xiongsy@localhost:~/icslab/xiongsy_Lab0$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
xiongsy@localhost:~/icslab/xiongsy_Lab0$
```

合并时，Git 会比较两个分支相对于共同祖先的修改。如果只有一方修改某一位位置，通常可以自动合并；当两边对同一位置作出不同修改且无法自动整合时，就会产生冲突。

在 main.c 中可以看见 <<<<<< HEAD、===== 和 >>>>>> feature，分别标记了产生冲突的两个不同版本。

冲突的解决：



```
icslab > xiongsy_Lab0 > C main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6     <<<<<< HEAD (当前更改)
7     printf("icslab0,xiongsy-test main分支\n");
8     =====
9     printf("feature branch\n");
10    >>>>>> feature (传入的更改)
11    printf("Hello, world!\n");
12 }
```

在 vscode 中，可以发现便捷的更改选择，可以选择不同的保留，根据需求选择即可解决冲突